

DCA-Trie Project Handbook

A Comprehensive Guide to the Codebase

Architecture, Metrics, Dependencies, and Workflows

Bernard

May 24, 2026

Contents

Preface	iv
1 Project Overview	1
1.1 Research Problem	1
1.2 The DCA-Trie Solution	1
1.3 Three Approaches	1
1.4 Directory Structure	2
2 Architecture	3
2.1 Layer Diagram	3
2.2 Module Dependency Graph	3
2.3 Data Flow: Prediction Pipeline	4
3 Core Library: src/	5
3.1 trie.py — Trie Implementations	5
3.1.1 Class Trie	5
3.1.2 Class MarisaTrie	5
3.1.3 Key Design Decision	5
3.2 graph_constrained_decoding.py — Token-Level Constraint	5
3.2.1 How It Works	6
3.2.2 Critical Detail	6
3.3 qa_prompt_builder.py — Prompt Construction	6
3.3.1 Class Hierarchy	6
3.3.2 Base Class: GraphConstrainedPromptBuilder	6
3.3.3 Prompt Templates	7
3.3.4 Path String Format	7
4 Model Layer: src/llms/	8
4.1 Architecture Overview	8
4.2 base_language_model.py — Abstract Base Class	8
4.3 base_hf_causal_model.py — HuggingFace Causal LM	8
4.3.1 Model Loading	8
4.3.2 Generation Modes	9
4.3.3 Generation Method	9
4.3.4 Prompt Preparation	9
4.4 graph_constrained_decoding_model.py — GCR Model	10
4.5 llm_proxy.py — FastChat/OpenAI Proxy	10
4.5.1 start_fastchat_api.py	10
4.6 conv_prompt.py — Conversation Templates	10
4.7 model_adapter.py — Prompt Adapters	11
4.8 chatgpt.py — OpenAI Direct	11
5 Utility Modules: src/utils/	12
5.1 graph_utils.py — Graph Operations	12
5.1.1 build_graph	12
5.1.2 dfs	12
5.1.3 get_truth_paths	12

5.1.4	bfs_with_rule	12
5.1.5	get_simple_paths	12
5.1.6	get_negative_paths / get_random_paths	12
5.2	qa_utils.py — Evaluation Metrics	13
5.2.1	Answer Normalisation	13
5.2.2	Per-Sample Metrics	13
5.2.3	Aggregate Evaluators	13
5.3	align_utils.py — Path Alignment	13
5.4	utils.py — File and String Utilities	14
5.5	training_utils.py	14
6	Symbolic TypeOracle	15
6.1	Location and Purpose	15
6.2	Type Constants	15
6.3	Answer Type Inference	15
6.4	Relation Schema	15
6.5	Class TypeOracle	16
6.5.1	Construction	16
6.5.2	Gate 1: Answer Type Gate (type_gate)	16
6.5.3	Gate 2: Property Range Gate (range_gate)	16
6.5.4	Combined Gate	16
6.5.5	SIR Computation Helper	17
7	Workflow Scripts	18
7.1	predict_paths_and_answers.py — GCR Baseline	18
7.1.1	Data Flow	18
7.1.2	Key Arguments	18
7.1.3	Prompt Builder Initialisation	18
7.1.4	Generation Call	19
7.2	predict_symbolic_dca_trie.py — Symbolic DCA-Trie	19
7.2.1	V1: Static Filtering (Algorithm 1)	19
7.2.2	V2: Step-Wise Expansion (Algorithm 2)	20
7.2.3	SIR Computation	20
7.3	predict_final_answer.py — Answer-Only Prediction	20
7.4	finetune_kg_specialized_llm.py — Training	20
7.4.1	Training Data Format	21
7.4.2	LoRA Configuration	21
7.5	build_graph_index.py and build_shortest_path_index.py	21
8	Metrics and Evaluation	22
8.1	Answer Metrics	22
8.1.1	Accuracy (Per-Sample)	22
8.1.2	Hit@1 (Per-Sample)	22
8.1.3	F1 Score	22
8.1.4	Aggregate Metrics	22
8.2	Path Metrics	23
8.2.1	Path F1	23
8.2.2	Path-Answer F1	23
8.2.3	Structural Faithfulness	23
8.3	Semantic Irrelevance Ratio (SIR)	23
8.3.1	Overall SIR	23
8.3.2	Decomposed SIR	23

8.3.3	Interpretation	23
8.4	False Negative Rate (FNR)	24
8.5	Hallucination Ratio	24
9	Dependencies and Environment	25
9.1	Critical Dependencies	25
9.2	Optional Dependencies	25
9.3	Environment Variables	25
9.4	Python Concepts	26
9.4.1	Model Registry Pattern	26
9.4.2	Factory Method with CLI Integration	26
9.4.3	Chat Template Pattern	26
9.4.4	Prefix-Allowed-Tokens Callback	26
9.4.5	Frozen Sets for Type Safety	27
9.4.6	Lazy Initialisation and Singleton Workers	27
9.5	Package Management	27
10	Experiment Workflows	28
10.1	Zero-Shot Baseline	28
10.2	GCR Baseline	28
10.3	DCA-Trie v1 (Symbolic)	28
10.4	DCA-Trie v2 (Step-Wise)	28
10.5	Graph Index Pre-Computation	29
10.6	Fine-Tuning a KG-Specialised LLM	29
11	CLI Reference	30
11.1	Global Arguments	30
11.2	Model Arguments	30
11.3	Decoding Strategies	30
11.4	DCA-Trie Specific Arguments	31
11.5	Training Arguments	31
12	Frequently Asked Questions	32
12.1	Architecture and Design	32
12.2	Evaluation	32
12.3	Troubleshooting	33
12.4	Performance	33
Appendix:	Quick Reference	34
12.5	File Function Map	34
12.6	Python Invocations Quick Reference	34

Preface

This handbook provides a comprehensive reference for the DCA-Trie (Domain-Constraint-Aware Trie) project – a graph-constrained reasoning framework for knowledge graph question answering (KGQA). The project implements and evaluates three approaches to reducing the path-explosion problem in KG-constrained decoding:

1. **Approach 1 (Cosine scorer):** Embedding-based path scoring using sentence transformers and cosine similarity.
2. **Approach 2 (Decomposed oracle):** Factorised scoring with separate relational relevance, entity type gate, and trajectory components.
3. **Approach 3 (Symbolic TypeOracle):** Pure ontology-based path pruning using Freebase schema triples — no embeddings, no thresholds, no GPU.

The handbook covers every Python module in the codebase, explaining its purpose, interfaces, data flow, and relationships to other modules. It also documents all evaluation metrics, CLI arguments, dependencies, and experiment workflows.

Chapter 1

Project Overview

1.1 Research Problem

Knowledge Graph Question Answering (KGQA) requires finding answer entities reachable via multi-hop reasoning paths in a KG. Formally, given a question q and a knowledge graph $\mathcal{K} = (\mathcal{E}, \mathcal{R}, T)$, find $A \subseteq \mathcal{E}$ such that there exists a reasoning path $e_0 \xrightarrow{r_1} e_1 \xrightarrow{r_2} \dots \xrightarrow{r_k} e_k$ where e_0 is a topic entity from q and $e_k \in A$ is the answer.

The core challenge for constrained-decoding approaches is the **path explosion problem**: a KG entity with average out-degree $\bar{d} = 20$ generates up to $\bar{d}^k = 8,000$ candidate paths at $k = 3$ hops. GCR (Luo et al., ICML 2025) builds a static trie over all paths, guaranteeing 100% structural faithfulness but at exponential memory cost.

1.2 The DCA-Trie Solution

DCA-Trie prunes the constraint set using KG ontology metadata — specifically, type and range declarations that are *already present in the KG* but unused by prior methods. The pruning is:

- **Symbolic**: Uses set lookups over type labels, not embedding similarity.
- **Deterministic**: No thresholds, no random seed dependence, no GPU.
- **Composable**: Each oracle gate can be independently enabled/disabled.
- **Provably safe**: False negative analysis confirms 0% gold-path exclusion.

1.3 Three Approaches

Table 1.1: Approach comparison

Property	Approach 1		Approach 2		Approach 3
Encoder	Sentence	trans-	Sentence	trans-	None
	former		former		
Threshold τ	Yes (0.25)		Yes (0.25)		None
Type checking	Implicit		Hard gate		Ontology-based
Range checking	None		None		Ontology-based
Per-path cost	Encoder	forward	Encoder	forward	O(1) set lookup
	pass		pass		
Deterministic	No (float noise)		No (float noise)		Yes

1.4 Directory Structure

```
graph-constrained-reasoning/  
+-- src/                                # Core library  
|   +-- llms/                          # Model wrappers (HF, proxy, GPT)  
|   +-- utils/                         # Graph, QA, training utilities  
|   +-- qa_prompt_builder.py           # Prompt construction & trie building  
|   +-- trie.py                       # Trie implementations  
|   +-- graph_constrained_decoding.py  # Token-level constraint logic  
+-- workflow/                         # Runnable experiment scripts  
+-- approach1_cosine/                 # Cosine scorer (standalone)  
+-- approach2_decomposed/             # Decomposed oracle (standalone)  
+-- approach3_symbolic/               # TypeOracle + notebooks  
+-- notebooks/                       # Jupyter experiments  
+-- paper/                           # LaTeX paper  
+-- kg-mini-book/                    # Supplementary LaTeX docs  
+-- results/                         # Experiment outputs  
+-- scripts/                         # Shell scripts  
+-- accelerate_configs/               # DeepSpeed configs  
+-- resources/                       # Figures  
+-- pyproject.toml                   # Project metadata  
+-- .env.example                     # Environment template
```

Chapter 2

Architecture

2.1 Layer Diagram

The codebase is organised into three layers:

```
Layer 3: Workflow Scripts
(predict_paths_and_answers.py, predict_symbolic_dca_trie.py,
 predict_final_answer.py, finetune_kg_specialized_llm.py)
    |
    | imports
    v
Layer 2: Core Library (src/)
(qa_prompt_builder.py, trie.py, graph_constrained_decoding.py)
    |
    | imports
    v
Layer 1: Foundation Modules
(src/llms/, src/utils/, approach3_symbolic/type_oracle.py)
```

2.2 Module Dependency Graph

```
workflow/predict_paths_and_answers.py
+-- src.llms.get_registered_model()      [model factory]
+-- src.qa_prompt_builder                [prompts + trie]
+-- src.utils.qa_utils.eval_path_result_w_ans [eval]
+-- src.utils (build_graph, dfs, path_to_string)

workflow/predict_symbolic_dca_trie.py
+-- src.llms.get_registered_model()
+-- src.qa_prompt_builder
+-- src.utils.qa_utils
+-- approach3_symbolic.type_oracle.TypeOracle
+-- src.graph_constrained_decoding.GraphConstrainedDecoding [v2 only]
+-- src.trie.MarisaTrie

src/qa_prompt_builder.py
+-- src.utils (build_graph, dfs, path_to_string)
+-- src.trie.MarisaTrie

src/llms/__init__.py -> factory
+-- base_language_model.py      [abstract base]
+-- base_hf_causal_model.py     [HF causal LM wrapper]
+-- graph_constrained_decoding_model.py [HF + trie]
+-- llm_proxy.py                [FastChat/OpenAI proxy]
```



```
+-- chatgpt.py           [OpenAI direct]
+-- conv_prompt.py       [chat templates]
+-- model_adapter.py     [prompt adapters]
```

2.3 Data Flow: Prediction Pipeline

```
Dataset (HF: rmanluo/RoG-webqsp)
|
| load_dataset("rmanluo/RoG-webqsp", split="test[:10]")
v
List of samples, each containing:
{question, answer, q_entity, a_entity, graph, id}
|
| For each sample:
v
1. Build NetworkX graph from graph triples
2. Enumerate candidate paths via DFS (depth = index_path_length)
3. [Optional] Apply TypeOracle gates to filter paths
4. Build MarisaTrie from surviving paths
5. Construct prompt template with question + entities
6. Call model.generate_sentence(input, trie, ...)
   |-- GraphConstrainedDecoding masks logits to trie-valid tokens
   |-- LLM generates path tokens under constraint
   v
7. Parse generated text into predictions
8. Evaluate against ground_truth answers (Hits@1, F1)
9. [Symbolic-only] Compute SIR metrics
```

Chapter 3

Core Library: src/

3.1 trie.py — Trie Implementations

This module provides three trie implementations for storing and querying tokenised path sequences.

3.1.1 Class Trie

A pure-Python trie using nested dictionaries. Each node maps a token to the next node:

```
1 class Trie:
2     def __init__(self):
3         self._next = {}          # token -> Trie node
4         self._is_end = False     # whether this node ends a complete sequence
5
6     def insert(self, sequence: List[int]) -> None
7     def get(self, prefix: List[int]) -> List[int]
```

`get(prefix)` traverses the trie following each token in `prefix`, then returns all tokens that validly continue from that point. Returns an empty list if the prefix is invalid.

3.1.2 Class MarisaTrie

Wrapper around `marisa-trie` (a C extension for efficient prefix tries). Used in production because it is orders of magnitude faster and more memory-efficient than the pure-Python version:

```
1 class MarisaTrie:
2     def __init__(self, sequences, max_token_id=None):
3         # sequences: list of list[int], each ending with eos_token_id
4
5     def get(self, prefix: List[int]) -> List[int]
6     def __len__(self) -> int
7     def __getitem__(self, idx: int) -> List[int]
```

3.1.3 Key Design Decision

The trie stores *sequences of token IDs* (not strings). Token sequences end with `eos_token_id`. The `get()` method returns all token IDs that can follow the given prefix, which the constrained decoding module uses to mask the LLM's vocabulary.

3.2 graph_constrained_decoding.py — Token-Level Constraint

This is the core mechanism that guarantees 100% structural faithfulness.

```
1 class GraphConstrainedDecoding:
2     def __init__(self, tokenizer, trie, start_token_ids=None,
3                   end_token_ids=None, enable_constrained_by_default=False):
4         self.all_tokens = list(range(len(tokenizer))) # full vocabulary
```

```

5  def allowed_tokens_fn(self, batch_id: int, sent: torch.Tensor):
6      # 1. Check if we've entered a <PATH>...</PATH> block
7      # 2. If inside: return trie.get(sent[L_input:]) as allowed tokens
8      # 3. If outside: return all_tokens (no constraint)
9      # 4. If trie has no valid continuation: fall back to all_tokens
10

```

3.2.1 How It Works

1. The `allowed_tokens_fn` is passed to HuggingFace's `model.generate()` as the `prefix_allowed_token` callback.
2. At each generation step, the callback checks whether the sequence is inside a constrained block (between `<PATH>` and `</PATH>` tokens).
3. If inside: it looks up the current generation prefix in the trie, and returns only the tokens that are valid continuations.
4. If the trie returns no valid tokens (empty list), it falls back to the full vocabulary — this prevents infinite loops.

3.2.2 Critical Detail

The trie is built from *path strings* tokenised *without* special tokens (via `add_special_tokens=False`). The generation prefix used for trie lookup excludes the input prompt length (`sent[L_input:]`), so only the generated portion is matched against the trie.

3.3 qa_prompt_builder.py — Prompt Construction

This module provides a hierarchy of six prompt builder classes:

3.3.1 Class Hierarchy

```

GraphConstrainedPromptBuilder (base)
+-- PathGenerationPromptBuilder      (path-only generation)
+-- JointReasoningPromptBuilder      (path + answer, with PATH_START/END)
|   +-- PathGenerationWithAnswerPromptBuilder (joint, used by GCR baseline)
+-- RetrievalPromptBuilder           (entity/relation/triple retrieval)
PromptBuilder                       (standalone final-answer builder)

```

3.3.2 Base Class: GraphConstrainedPromptBuilder

Core methods:

```

1  class GraphConstrainedPromptBuilder:
2      def __init__(self, tokenizer, prompt="zero-shot",
3                  undirected=False, index_path_length=2, add_rule=False):
4          # Stores config; loads prompt template by name
5
6      def get_graph_index(self, question_dict):
7          # 1. If question_dict has "paths": use them
8          # 2. Else: build NetworkX graph, run DFS from q_entity
9          # 3. Tokenise path strings, add eos_token_id
10         # 4. Return MarisaTrie(tokenized_paths)
11
12     def process_input(self, question_dict, return_trie=True):
13         # 1. Extract question, start_node, answer_node, choices

```

```

14         # 2. Build trie via get_graph_index()
15         # 3. Find ground-truth paths via get_truth_paths()
16         # 4. Format prompt template with question + entities
17         # 5. Return (formatted_input, ground_paths, trie)

```

3.3.3 Prompt Templates

The `JointReasoningPromptBuilder` uses a zero-shot template:

```

ZERO_SHOT_PROMPT = """Reasoning path is a sequence of triples in the KG
that connects the topic entities in the question to answer entities.
It should start with <PATH> and end with </PATH>.
When given a question, please generate some reasoning paths in the KG
starting from the topic entities that you believe can aid in answering it.
Then, use these reasoning paths to derive the answer to the question.

# Question:
{question}
# Topic entities:
{entities}
"""

```

The `<PATH>` and `</PATH>` special tokens (defined as `PATH_START_TOKEN` and `PATH_END_TOKEN`) act as delimiters that the `GraphConstrainedDecoding` module uses to enter/exit constrained mode.

3.3.4 Path String Format

Paths are serialised using the arrow format:

```

Elvis_Presley -> people.person.place_of_birth -> Tupelo
Elvis_Presley -> people.person.parents -> Vernon_Presley -> people.person.
    children -> Elvis_Presley

```

The trie is built by tokenising these strings (without special tokens) and appending `eos_token_id`.

Chapter 4

Model Layer: src/llms/

4.1 Architecture Overview

The llms package implements a model-registry pattern. All model classes inherit from BaseLanguageModel and are registered in __init__.py.

```
1 registered_language_models = {
2     'gpt': ChatGPT,
3     'others': HfCausalModel,
4     'gcr': GraphConstrainedDecodingModel,
5     'proxy': LLMPProxy,
6 }
7
8 def get_registered_model(model_name) -> BaseLanguageModel:
9     for key, value in registered_language_models.items():
10         if key in model_name.lower():
11             return value
12     return HfCausalModel # default
```

The model name is checked as a substring: "gcr-Llama-2-7b-chat-hf" matches "gcr" and returns GraphConstrainedDecodingModel.

4.2 base_language_model.py — Abstract Base Class

Defines the interface:

```
1 class BaseLanguageModel:
2     @staticmethod
3     def add_args(parser) # Add CLI args
4     def load_model(self) # Load model weights
5     def token_len(self, text) # Count tokens
6     def prepare_for_inference(self) # Move to eval mode
7     def prepare_model_prompt(self, input) # Apply chat template
8     def generate_sentence(self, input, trie, **kwargs) # Generate
```

4.3 base_hf_causal_model.py — HuggingFace Causal LM

The most complex module (~212 lines). Key features:

4.3.1 Model Loading

```
1 class HfCausalModel(BaseLanguageModel):
2     def load_model(self):
3         self.tokenizer = AutoTokenizer.from_pretrained(self.model_path)
4         self.model = AutoModelForCausalLM.from_pretrained(
5             self.model_path,
6             torch_dtype=torch.bfloat16,
7             attn_implementation=attn_impl,
8             quantization_config=...,
```

```

9         device_map="auto",
10     )

```

Supports four dtypes (fp32, fp16, bf16) and two quantisation modes (4bit, 8bit via bitsandbytes).

4.3.2 Generation Modes

Five decoding strategies are supported:

```

1  if mode == "greedy":
2      generation_config.num_beams = 1
3      generation_config.do_sample = False
4  elif mode == "beam":
5      generation_config.num_beams = k
6      generation_config.do_sample = False
7  elif mode == "sampling":
8      generation_config.do_sample = True
9      generation_config.temperature = 0.6
10     generation_config.top_p = 0.95
11  elif mode == "group-beam":
12     generation_config.num_beams = k
13     generation_config.num_beam_groups = k
14     generation_config.diversity_penalty = 1.0
15     generation_config.do_sample = False
16  elif mode == "beam-early-stopping":
17     generation_config.num_beams = k
18     generation_config.early_stopping = True

```

4.3.3 Generation Method

The main generation method handles constrained decoding via the `prefix_allowed_tokens_fn` callback:

```

1  def generate_sentence(self, input, trie, start_token_ids=None,
2                      end_token_ids=None, enable_constrained_by_default=
3                      False):
4      gcr = GraphConstrainedDecoding(
5          self.tokenizer, trie, start_token_ids, end_token_ids,
6          enable_constrained_by_default
7      )
8      inputs = self.tokenizer(input, return_tensors="pt",
9                             add_special_tokens=False).to(self.model.device)
9      outputs = self.model.generate(
10         **inputs,
11         generation_config=self.generation_cfg,
12         prefix_allowed_tokens_fn=gcr.allowed_tokens_fn,
13         ...
14     )
15     return self.tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
16                                skip_special_tokens=True)

```

4.3.4 Prompt Preparation

The `prepare_model_prompt` method applies the appropriate chat template:

```

1  def prepare_model_prompt(self, input):
2      if self.chat_model and self.conv:
3          # For Group-beam and beam modes: apply system + user template
4          self.conv.messages = []
5          self.conv.append_message(self.conv.roles[0], input)

```

```

6         self.conv.append_message(self.conv.roles[1], None)
7         return self.conv.get_prompt()
8     else:
9         return input

```

4.4 graph_constrained_decoding_model.py — GCR Model

A thin subclass of `HfCausalModel` that overrides `generate_sentence` to ensure the `GraphConstrainedDecoding` callback is properly wired. This is what gets instantiated when the model name contains "gcr".

4.5 llm_proxy.py — FastChat/OpenAI Proxy

Used when the model name contains "proxy". Connects to a FastChat API server (either locally launched or remote):

```

1 class LLMProxy(BaseLanguageModel):
2     def __init__(self, args):
3         self.conv_template = args.conv_template
4         self.client = OpenAI(base_url=f"http://{host}:{port}/v1",
5                               api_key="EMPTY")
6
7     def generate_sentence(self, input, trie=None, **kwargs):
8         # FastChat doesn't support constrained decoding natively,
9         # so trie is ignored (standard LLM response, no mask)
10        response = self.client.chat.completions.create(
11            model=self.model_name,
12            messages=[{"role": "user", "content": input}],
13            max_tokens=self.max_new_tokens,
14            temperature=0,
15        )
16        return response.choices[0].message.content

```

4.5.1 start_fastchat_api.py

Launches a local FastChat stack as subprocesses:

1. Controller (port 21001)
2. Model worker (loads the model, registers with controller)
3. OpenAI-compatible API server (port 8000)

Uses `atexit.register(cleanup)` to kill all subprocesses on exit.

4.6 conv_prompt.py — Conversation Templates

A 1,244-line module defining ~40 conversation templates for different chat models. Each template is a `Conversation` dataclass:

```

1 @dataclass
2 class Conversation:
3     system: str                # System prompt
4     roles: List[str]           # ["user", "assistant"]
5     messages: List[List[str]]  # [[role, message], ...]
6     offset: int
7     sep_style: SeparatorStyle  # How to format the prompt

```

```

8     sep: str                # Separator string
9     sep2: str               # Secondary separator
10
11     def get_prompt(self) -> str: # Render to string
12     def append_message(self, role, message): # Add turn
13     def copy(self):            # Deep copy

```

Registered templates include: Vicuna v0/v1/v1.1/v1.2, Llama-2, Qwen-7B, Mistral, Alpaca, ChatGLM, Falcon, Baichuan, etc.

4.7 model_adapter.py — Prompt Adapters

Wraps HfCausalModel with specific conversation templates:

```

1 class LlamaAdapter(HfCausalModel):
2     def __init__(self, args):
3         super().__init__(args)
4         self.conv = conv_templates["llama-2"].copy()
5
6 class QwenAdapter(HfCausalModel):
7     def __init__(self, args):
8         super().__init__(args)
9         self.conv = conv_templates["qwen-7b-chat"].copy()

```

4.8 chatgpt.py — OpenAI Direct

Wraps the OpenAI API with retry logic:

```

1 class ChatGPT(BaseLanguageModel):
2     def generate_sentence(self, input, trie=None, **kwargs):
3         for attempt in range(self.retry):
4             response = self.client.chat.completions.create(
5                 model=self.model_path or "gpt-4",
6                 messages=[{"role": "user", "content": input}],
7                 max_tokens=1024,
8             )
9             return response.choices[0].message.content

```


Chapter 5

Utility Modules: `src/utls/`

5.1 `graph_utils.py` — Graph Operations

Core graph operations using NetworkX.

5.1.1 `build_graph`

```
1 def build_graph(graph: list, undirected=False) -> nx.DiGraph | nx.Graph:
2     G = nx.Graph() if undirected else nx.DiGraph()
3     for h, r, t in graph:
4         G.add_edge(h.strip(), t.strip(), relation=r.strip())
5     return G
```

Each triple `[head, relation, tail]` becomes a directed edge with a `relation` attribute.

5.1.2 `dfs`

```
1 def dfs(graph, start_node_list, max_length):
2     # Enumerate all paths up to max_length from start_node_list
3     # Uses recursion with set-based deduplication
4     # Returns List[List[Tuple[head, relation, tail]]]
```

Time complexity: $O(\bar{d}^k)$ in the worst case, where $\bar{d}$ is average out-degree and k is `max_length`.

5.1.3 `get_truth_paths`

```
1 def get_truth_paths(q_entity, a_entity, graph):
2     # Uses nx.all_shortest_paths between each (q_entity, a_entity) pair
3     # Converts node sequences to (head, relation, tail) triples
4     # No hop limit -- uses actual shortest paths
```

5.1.4 `bfs_with_rule`

```
1 def bfs_with_rule(graph, start_node, target_rule, max_p=10):
2     # BFS that only follows edges matching target_rule at each depth
3     # target_rule: list of relation strings
4     # Used when rules are predicted and must be followed exactly
```

5.1.5 `get_simple_paths`

Like `get_truth_paths` but with a configurable hop cutoff and using `nx.all_simple_edge_paths`.

5.1.6 `get_negative_paths` / `get_random_paths`

Generate negative samples via random walks for training. Uses the `walker` library.

5.2 qa_utils.py — Evaluation Metrics

5.2.1 Answer Normalisation

```

1 def normalize(s: str) -> str:
2     # Lowercase, remove punctuation, remove articles (a/an/the),
3     # remove <pad> tokens, collapse whitespace
4     return normalized_string
5
6 def match(s1: str, s2: str) -> bool:
7     # Returns True if normalized s2 appears as substring of normalized s1

```

5.2.2 Per-Sample Metrics

```

1 def eval_acc(prediction, answer):
2     # Accuracy = fraction of answer strings matched in prediction
3     matched = sum(1 for a in answer if match(prediction, a))
4     return matched / len(answer)
5
6 def eval_hit(prediction, answer):
7     # Hit = 1 if ANY answer matched, else 0
8     return 1 if any(match(prediction, a) for a in answer) else 0
9
10 def eval_f1(prediction_list, answer_list):
11     # F1 = 2 * precision * recall / (precision + recall)
12     # precision = correct_predictions / len(prediction)
13     # recall = matched_answers / len(answer)

```

5.2.3 Aggregate Evaluators

Each aggregate function reads a `predictions.jsonl` file, computes per-sample metrics, and writes both a detailed JSONL output and a summary text file:

eval_result Basic accuracy/hit/F1/precision/recall.

eval_rank_results Top-k metrics (Accuracy@k, Hit@k, F1@k).

eval_path_result Path-level metrics (Path F1).

eval_path_result_w_ans Joint path+answer metrics.

eval_joint_result Parses “Reasoning path:” / “the answer is:” markers.

5.3 align_utils.py — Path Alignment

Used during training to evaluate rule-sequence prediction quality:

```

1 def compute_metrics(preds, labels, rel_dict):
2     # Hallucination = relation not in rel_dict
3     # Hit, Precision, Recall, F1 on rule sequences joined by <SEP>
4
5 def eval_generation(result_path, rel_dict_path, debug=False):
6     # Reads predictions, computes aggregate metrics

```

5.4 utils.py — File and String Utilities

```
1 def path_to_string(path):  
2     # [(h1, r1, t1), (h2, r2, t2)] -> "h1 -> r1 -> t1 -> r2 -> t2"  
3  
4 def load_jsonl(path):  
5     # Reads a JSONL file, returns list of parsed dicts  
6  
7 def read_prompt(prompt_path):  
8     # Reads a prompt template from a file
```

5.5 training_utils.py

```
1 def resize_tokenizer_and_embedding(tokenizer, model, new_tokens):  
2     # Adds new tokens to tokenizer, resizes model embeddings
```

Chapter 6

Symbolic TypeOracle

6.1 Location and Purpose

File: `approach3_symbolic/type_oracle.py` (586 lines). This is the core of Approach 3 (the symbolic DCA-Trie). It requires no external dependencies beyond Python stdlib — no PyTorch, no sentence-transformers, no GPU.

6.2 Type Constants

The module defines nine frozen sets of Freebase human-readable type names:

```
1 _PERSON_TYPES      = frozenset({"Person", "Politician", "Author", ...})
2 _LOCATION_TYPES      = frozenset({"Location", "Country", "City/Town/Village",
3 ...})
4 _ORGANIZATION_TYPES = frozenset({"Organization", "Company", ...})
5 _CREATIVE_WORK_TYPES = frozenset({"Film", "Book", "TV Program", ...})
6 _DATE_TYPES         = frozenset({"Date/Time"})
7 _LANGUAGE_TYPES     = frozenset({"Human Language"})
8 _AWARD_TYPES         = frozenset({"Award", "Award honor"})
9 _PROFESSION_TYPES   = frozenset({"Profession"})
10 _GENRE_TYPES        = frozenset({"TV Genre", "Music Genre", "Film Genre",
11 ...})
```

These are extracted from `common.topic.notable_types` triples in the WebQSP sub-graphs.

6.3 Answer Type Inference

The `ANSWER_TYPE_MAP` dictionary maps question keywords to expected answer types:

```
1 ANSWER_TYPE_MAP = {
2     "who": _PERSON_TYPES,
3     "where": _LOCATION_TYPES,
4     "when": _DATE_TYPES,
5     "movie": _CREATIVE_WORK_TYPES,
6     "award": _AWARD_TYPES,
7     "company": _ORGANIZATION_TYPES,
8     ...
9 }
```

The `_QUESTION_PATTERNS` list contains 19 regex patterns for matching these keywords, with quoted entity mentions masked to avoid false matches.

6.4 Relation Schema

The `_RELATION_SCHEMA` dictionary maps ~40 Freebase relation IDs to their domain and range type constraints:

```

1 _RELATION_SCHEMA = {
2     "people.person.place_of_birth": {
3         "domain": _PERSON_TYPES,
4         "range": _LOCATION_TYPES,
5     },
6     "film.film.directed_by": {
7         "domain": _CREATIVE_WORK_TYPES,
8         "range": _PERSON_TYPES,
9     },
10    ...
11 }

```

6.5 Class TypeOracle

6.5.1 Construction

```

1 @classmethod
2 def from_graph(cls, graph_triples):
3     # Iterates over graph triples, extracting types from:
4     #   - common.topic.notable_types
5     #   - freebase.type_hints.included_types (excluding "Topic" supertype)
6     #   - freebase.type_profile.strict_included_types
7     # Builds Dict[entity_name -> FrozenSet[type_name]]
8     return cls(entity_type_map=frozen)

```

6.5.2 Gate 1: Answer Type Gate (type_gate)

Only applies at the terminal hop of a path:

```

1 def type_gate(self, entity_name, answer_types, hop, max_hop):
2     if hop < max_hop:         return True # non-terminal: always allow
3     if not answer_types:      return True # no constraint inferred
4     etypes = self._entity_types.get(entity_name, frozenset())
5     if not etypes:           return True # unknown type: allow
6     return bool(etypes & answer_types) # must intersect

```

6.5.3 Gate 2: Property Range Gate (range_gate)

Applies at every hop:

```

1 def range_gate(self, relation, tail_entity):
2     rel_schema = self._schema.get(relation)
3     if rel_schema is None: return True # unknown relation: allow
4     range_types = rel_schema.get("range", frozenset())
5     if not range_types:    return True # no range constraint: allow
6     etypes = self._entity_types.get(tail_entity, frozenset())
7     if not etypes:         return True # unknown entity: allow
8     return bool(etypes & range_types) # must intersect

```

6.5.4 Combined Gate

```

1 def is_admissible(self, relation, tail_entity, answer_types, hop, max_hop):
2     return (self.type_gate(tail_entity, answer_types, hop, max_hop)
3             and self.range_gate(relation, tail_entity))

```

6.5.5 SIR Computation Helper

```
1 def compute_type_irrelevance(paths, oracle, answer_types, max_hop):  
2     # SIR*_type = fraction of paths whose terminal entity type  
3     # is incompatible with inferred answer types
```

Chapter 7

Workflow Scripts

7.1 predict_paths_and_answers.py — GCR Baseline

This is the primary GCR baseline script (184 lines).

7.1.1 Data Flow

1. Load dataset via `datasets.load_dataset(path, split=split)`.
2. (Optional) Merge with pre-computed rule predictions.
3. Instantiate model via `get_registered_model(args.model_name)`.
4. Build `PathGenerationWithAnswerPromptBuilder`.
5. For each sample:
 - (a) `prompt_builder.process_input(data) → (input, ground_paths, trie)`.
 - (b) `model.prepare_model_prompt(input) → formatted prompt`.
 - (c) `model.generate_sentence(input, trie, start_ids, end_ids)`.
 - (d) Write result to `predictions.jsonl`.
6. Run `eval_path_result_w_ans(predictions.jsonl)`.

7.1.2 Key Arguments

```
--model_name    gcr-Llama-2-7b-chat-hf  (determines model class via registry)
--d             RoG-webqsp              (dataset name)
--split         test[:100]              (split with optional slice)
--index_path_length 2                    (max DFS depth = k)
--generation_mode group-beam            (decoding strategy)
--k             5                       (beam size / number of paths)
--prompt_mode   zero-shot               (prompt template)
```

7.1.3 Prompt Builder Initialisation

```
1 input_builder = PathGenerationWithAnswerPromptBuilder(
2     model.tokenizer, args.prompt_mode,
3     index_path_length=args.index_path_length,
4     undirected=args.undirected,
5     add_rule=args.add_rule,
6 )
```

7.1.4 Generation Call

```

1 start_token_ids = model.tokenizer.convert_tokens_to_ids("<PATH>")
2 end_token_ids = model.tokenizer.convert_tokens_to_ids("</PATH>")
3 prediction = model.generate_sentence(
4     input, trie,
5     start_token_ids=start_token_ids,
6     end_token_ids=end_token_ids,
7     enable_constrained_by_default=False,
8 )

```

7.2 predict_symbolic_dca_trie.py — Symbolic DCA-Trie

The new symbolic approach (625 lines). Supports two modes:

7.2.1 V1: Static Filtering (Algorithm 1)

1. Build `TypeOracle.from_graph(data["graph"])`.
2. Infer `answer_types` from question via regex patterns.
3. Enumerate paths via DFS (same as GCR).
4. For each path:
 - (a) Check `range_gate` on every hop.
 - (b) Check `type_gate` on terminal hop only.
5. Build trie from surviving paths.
6. Run standard constrained decoding with the filtered trie.

Key function:

```

1 def build_symbolically_filtered_trie(question_dict, tokenizer,
2                                     index_path_length, undirected):
3     oracle = TypeOracle.from_graph(question_dict["graph"])
4     answer_types = oracle.infer_answer_types(question_dict["question"])
5     paths_list = utils.dfs(g, entities, index_path_length)
6
7     kept = []
8     for p in paths_list:
9         if all(oracle.range_gate(rel, t) for _, rel, t in p):
10             terminal = p[-1][2]
11             if oracle.type_gate(terminal, answer_types, len(p),
12                                index_path_length):
13                 kept.append(p)
14
15     kept_str = [path_to_string(p) for p in kept]
16     trie = build_trie_from_paths(kept_str, tokenizer, tokenizer.
17                                  eos_token_id)
18     return trie, kept_str, oracle, ...

```


7.2.2 V2: Step-Wise Expansion (Algorithm 2)

No upfront path enumeration. Instead:

1. Start with first-hop neighbours of the topic entity, filtered by `range_gate`.
2. Build a small trie of these single-hop paths.
3. Run constrained generation for one step.
4. Extract the committed entity from the model output.
5. Query its neighbours, gate them, build a new trie.
6. Repeat until end token or max hops reached.

7.2.3 SIR Computation

```

1 def compute_sir_metrics(paths_before, paths_after,
2                           n_range_blocked, n_type_blocked, ...):
3     sir = (paths_before - paths_after) / paths_before
4     sir_type = n_type_blocked / paths_before
5     sir_traj = n_range_blocked / paths_before
6     return {"sir": sir, "sir_type": sir_type, "sir_traj": sir_traj, ...}
7
8 def compute_false_negative_rates(dataset, index_path_length, undirected):
9     # For each gold path, check if type_gate or range_gate would exclude it
10    # Returns FNR_type and FNR_range

```

7.3 predict_final_answer.py — Answer-Only Prediction

Runs final answer prediction using a configured LLM (341 lines). Supports:

- **No-path mode:** Direct question answering (zero-shot baseline).
- **Rule-based:** Applies predicted relation rules via BFS.
- **Path-based:** Feeds predicated paths as context.
- **GCR mode:** Constrained decoding for answer generation.
- **Multi-process:** Pool-based parallel inference.
- **Simple/edge graph:** Alternative context formats.

7.4 finetune_kg_specialized_llm.py — Training

Supervised fine-tuning script (247 lines) using `trl.SFTTrainer`:

```

1 class ScriptArguments:
2     data_path_list: List[str]      # Training data directories
3     model_name_or_path: str        # Base model
4     use_peft: bool                 # Enable LoRA
5     lora_r: int = 8
6     lora_alpha: float = 16
7     lora_dropout: float = 0.05
8     load_in_4bit: bool = False
9     load_in_8bit: bool = False
10    n_path_per_sample: int = 10
11    response_template: str = "[/INST]"

```

7.4.1 Training Data Format

Each training sample is a question paired with sampled reasoning paths:

```
{
  "question": "what award did elvis presley win in 1971?",
  "paths": [
    "Elvis_Presley -> award.award_winner.awards_won -> Grammy_Award ...",
    ...
  ],
  "answer": ["Grammy Award"]
}
```

The `DataCollatorForCompletionOnlyLM` masks the question tokens, so the loss is computed only on the path and answer tokens.

7.4.2 LoRA Configuration

```
1 peft_config = LoraConfig(
2     r=args.lora_r, lora_alpha=args.lora_alpha,
3     lora_dropout=args.lora_dropout,
4     target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
5     bias="none", task_type="CAUSAL_LM",
6 )
```

7.5 build_graph_index.py and build_shortest_path_index.py

Pre-computation scripts that run DFS / shortest-path enumeration and save the results to disk via `datasets.save_to_disk`. This avoids re-running expensive path enumeration during training or inference.

Chapter 8

Metrics and Evaluation

8.1 Answer Metrics

These compare the model’s predicted answer string(s) against ground-truth answer strings.

8.1.1 Accuracy (Per-Sample)

Definition 8.1 (Answer Accuracy). For a single sample, let $\mathcal{A}_{\text{pred}}$ be the prediction string and $\mathcal{A}_{\text{gt}}$ be the set of ground-truth answer strings.

$$\text{Acc} = \frac{1}{|\mathcal{A}_{\text{gt}}|} \sum_{a \in \mathcal{A}_{\text{gt}}} \mathbb{1}[a \subseteq_{\text{norm}} \mathcal{A}_{\text{pred}}]$$

where $\subseteq_{\text{norm}}$ is substring matching after normalisation (lowercase, strip punctuation, remove articles, collapse whitespace).

8.1.2 Hit@1 (Per-Sample)

Definition 8.2 (Hit@1).

$$\text{Hit} = \mathbb{1} \left[\bigvee_{a \in \mathcal{A}_{\text{gt}}} a \subseteq_{\text{norm}} \mathcal{A}_{\text{pred}} \right]$$

Returns 1 if any ground-truth answer is found in the prediction, 0 otherwise.

8.1.3 F1 Score

Definition 8.3 (Answer F1). When predictions are a list of strings (parsed from multi-line output), compute:

$$\begin{aligned} \text{Precision} &= \frac{|\{\text{predicted answers that match any ground truth}\}|}{|\text{predicted}|} \\ \text{Recall} &= \frac{|\{\text{ground-truth answers found in any prediction}\}|}{|\text{ground truth}|} \\ \text{F1} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

8.1.4 Aggregate Metrics

For an entire dataset, metrics are averaged over all samples:

$$\text{Hits@1} = \frac{1}{N} \sum_{i=1}^N \text{Hit}_i$$

$$\overline{\text{F1}} = \frac{1}{N} \sum_{i=1}^N \text{F1}_i$$

8.2 Path Metrics

These evaluate the quality of generated KG reasoning paths.

8.2.1 Path F1

Compares generated paths (as strings) against ground-truth paths using the same F1 formula but where “answers” are path strings.

8.2.2 Path-Answer F1

A compound metric: paths and answers are parsed jointly from the model output. The path portion is compared against ground-truth paths, and the answer portion against ground-truth answers.

8.2.3 Structural Faithfulness

For constrained-decoding methods (GCR, DCA-Trie), this is always 100%: every generated path is guaranteed to exist in the KG because the trie is built from KG-valid paths only.

8.3 Semantic Irrelevance Ratio (SIR)

Introduced by the DCA-Trie project to measure how many semantically irrelevant paths remain in the constraint set after pruning.

8.3.1 Overall SIR

Definition 8.4 (SIR).

$$\text{SIR} = \frac{|\mathcal{P}_{\text{full}}| - |\mathcal{P}_{\text{pruned}}|}{|\mathcal{P}_{\text{full}}|} = \frac{N_{\text{pruned}}}{N_{\text{full}}}$$

where $\mathcal{P}_{\text{full}}$ is the set of all KG-valid paths (the GCR constraint set) and $\mathcal{P}_{\text{pruned}}$ is the set after DCA-Trie gates are applied.

8.3.2 Decomposed SIR

$$\begin{aligned} \text{SIR}_{\text{type}} &= \frac{N_{\text{type-blocked}}}{N_{\text{full}}} \\ \text{SIR}_{\text{traj}} &= \frac{N_{\text{range-blocked}}}{N_{\text{full}}} \end{aligned}$$

8.3.3 Interpretation

- $\text{SIR} = 0$ means no paths were pruned (no improvement over GCR).
- $\text{SIR} = 0.40$ means 40% of paths were pruned (constraint set reduced).
- High SIR_{type} indicates many terminal entities have incompatible types with the question.
- High SIR_{traj} indicates many intermediate hops violate range constraints.

8.4 False Negative Rate (FNR)

Measures the fraction of gold-truth paths that would be excluded by each gate:

Definition 8.5 (FNR).

$$\text{FNR}_{\text{gate}} = \frac{N_{\text{gold paths excluded by gate}}}{N_{\text{total gold paths}}}$$

A safe gate should have $\text{FNR} = 0$ (never exclude a valid answer path). The DCA-Trie TypeOracle achieves 0% FNR on both gates across WebQSP.

8.5 Hallucination Ratio

From `align_utils.py`: fraction of generated relation tokens that are not present in the KG’s relation vocabulary. Relevant for non-constrained methods. For constrained decoding, this is always 0.

Chapter 9

Dependencies and Environment

9.1 Critical Dependencies

Table 9.1: Critical dependencies

Package	Version	Role
<code>torch</code>	≥ 2.0	Core tensor operations, model inference
<code>transformers</code>	4.44.x	HuggingFace model loading, tokenization, generation pipeline
<code>accelerate</code>	$\geq 0.30.1$	Multi-GPU distributed inference/training
<code>datasets</code>	$\geq 2.19.2$	Dataset loading (HuggingFace hub), saving/loading indexes
<code>marisa-trie</code>	$\geq 1.2.0$	Efficient prefix trie (C extension) for constrained decoding
<code>networkx</code>	any	Graph construction, DFS, shortest paths
<code>scikit-learn</code>	$\geq 1.5.0$	<code>cosine_similarity</code> , <code>precision_score</code>
<code>peft</code>	$\geq 0.11.1$	LoRA/QLoRA for efficient fine-tuning
<code>trl</code>	$\geq 0.9.4$	SFTTrainer with completion-only LM collator
<code>bitsandbytes</code>	$\geq 0.43.1$	4-bit/8-bit quantisation (optional)

9.2 Optional Dependencies

Table 9.2: Optional dependencies

Package	Version	Role
<code>deepspeed</code>	$\geq 0.14.2$	ZeRO-3 optimisation for large models
<code>sentence-transformers</code>	any	Embedding models for Approach 1/2
<code>openai</code>	$\geq 1.31.1$	OpenAI API client (GPT-4/3.5)
<code>tiktoken</code>	$\geq 0.7.0$	Fast token counting for GPT models
<code>wandb</code>	$\geq 0.17.1$	Experiment logging (Weights & Biases)
<code>sentencepiece</code>	$\geq 0.2.0$	Tokenization for some models
<code>walker</code>	any	Random walks for negative sampling
<code>python-dotenv</code>	$\geq 1.0.1$	Loading <code>.env</code> for API keys

9.3 Environment Variables

Table 9.3: Environment variables

Variable	Default	Purpose
OPENAI_API_KEY	sk-xx	API key for OpenAI/GPT access
HF_TOKEN	hf_xxx	HuggingFace authentication (for gated models)
OPENAI_ORG	(none)	OpenAI organisation ID
OPENAI_BASE_URL	(none)	Custom API base URL (for FastChat proxies)
SLURM_CPUS_PER_TASK	(none)	CPU count override for Slurm environments

9.4 Python Concepts

The codebase demonstrates several important Python patterns:

9.4.1 Model Registry Pattern

`src/llms/__init__.py` implements a registry where model classes map to name sub-strings. This enables selecting inference backends via a single CLI argument:

```

1 def get_registered_model(model_name):
2     for key, value in registered_language_models.items():
3         if key in model_name.lower():
4             return value
5     return HfCausalModel # fallback

```

Usage: `gcr-llama-2-7b-chat-hf` → `GraphConstrainedDecodingModel`, `proxy-qwen2.5-3b` → `LLMProxy`.

9.4.2 Factory Method with CLI Integration

Model classes expose both constructor and CLI configuration via `add_args`:

```

1 LLM = get_registered_model(args.model_name)
2 LLM.add_args(parser) # Each model registers its own args
3 args = parser.parse_args() # Re-parse with new args registered
4 model = LLM(args) # Construct with parsed args

```

9.4.3 Chat Template Pattern

`conv_prompt.py` implements a builder-pattern for chat templates:

```

1 conv = Conversation(
2     system="You are a helpful AI assistant.",
3     roles=["USER", "ASSISTANT"],
4     sep_style=SeparatorStyle.ADD_COLON_TWO,
5     sep=" ", sep2="</s>",
6 )
7 conv.append_message(conv.roles[0], "What is the capital of France?")
8 conv.append_message(conv.roles[1], None)
9 prompt = conv.get_prompt()
10 # "USER: What is the capital of France?\nASSISTANT:"

```

9.4.4 Prefix-Allowed-Tokens Callback

A HuggingFace generation callback that constrains the vocabulary at each step:

```

1 model.generate(
2     ...,
3     prefix_allowed_tokens_fn=lambda batch_id, sent: allowed_tokens,
4 )

```

The callback receives the batch index and the current token sequence, and returns a list of allowed token IDs for the next step.

9.4.5 Frozen Sets for Type Safety

The TypeOracle uses `FrozenSet[str]` for immutable type collections. This enables $O(1)$ set intersection checks with hashing:

```

1 return bool(etypes & answer_types) # 0(min(|etypes|, |answer_types|))

```

9.4.6 Lazy Initialisation and Singleton Workers

`start_fastchat_api.py` launches compute-heavy subprocesses lazily and cleans them up with `atexit`:

```

1 import atexit, subprocess
2
3 def start_api():
4     controller = subprocess.Popen([...])
5     worker = subprocess.Popen([...])
6     server = subprocess.Popen([...])
7     atexit.register(lambda: (controller.kill(), worker.kill(), server.kill()))
8     return server

```

9.5 Package Management

The project uses **Poetry** for dependency management. Key details from `pyproject.toml`:

```

[tool.poetry.dependencies]
python = ">=3.10"
torch = ">=2.0"
transformers = "4.44.2"
datasets = ">=2.19.2"
accelerate = ">=0.30.1"
peft = ">=0.11.1"
trl = ">=0.9.4"
marisa-trie = ">=1.2.0"
scikit-learn = ">=1.5.0"
networkx = "*"
openai = ">=1.31.1"
tiktoken = ">=0.7.0"
wandb = ">=0.17.1"

```

To install:

```

poetry install
# or: pip install -r requirements.txt

```


Chapter 10

Experiment Workflows

10.1 Zero-Shot Baseline

```
python workflow/predict_final_answer.py \  
    --model_name proxy-qwen2.5-3b \  
    --d RoG-webqsp \  
    --split "test[:10]"
```

Uses LLMProxy to connect to a FastChat server running Qwen2.5-3B. No path generation or constrained decoding.

10.2 GCR Baseline

```
python workflow/predict_paths_and_answers.py \  
    --model_name gcr-qwen2.5-3b \  
    --d RoG-webqsp \  
    --split "test[:10]" \  
    --index_path_length 2 \  
    --generation_mode group-beam \  
    --k 5 \  
    --prompt_mode zero-shot
```

Builds a full KG trie, runs constrained decoding with `GraphConstrainedDecodingModel`. The model name contains "gcr" so the GCR model class is used.

10.3 DCA-Trie v1 (Symbolic)

```
python workflow/predict_symbolic_dca_trie.py \  
    --model_name gcr-qwen2.5-3b \  
    --d RoG-webqsp \  
    --split "test[:10]" \  
    --dca_mode v1 \  
    --index_path_length 2 \  
    --generation_mode group-beam \  
    --k 5 \  
    --prompt_mode zero-shot
```

Same as GCR, but paths are filtered through TypeOracle gates before trie construction. Reports SIR and FNR metrics.

10.4 DCA-Trie v2 (Step-Wise)

```
python workflow/predict_symbolic_dca_trie.py \  
    --model_name gcr-qwen2.5-3b \  
    --d RoG-webqsp \  
    --split "test[:10]" \  
    --dca_mode v2 \  
    --index_path_length 2
```

Custom generation loop that expands the trie dynamically. V2 does not use the standard `generate_sentence` interface; it calls `model.model.generate()` directly with a per-step trie update.

10.5 Graph Index Pre-Computation

```
python workflow/build_graph_index.py \  
  --data_path rmanluo \  
  --d RoG-webqsp \  
  --split test \  
  --K 2
```

Useful for large-scale experiments: pre-compute all DFS paths and save to disk, avoiding repeated computation across runs.

10.6 Fine-Tuning a KG-Specialised LLM

```
python workflow/finetune_kg_specialized_llm.py \  
  --model_name_or_path meta-llama/Llama-2-7b-chat-hf \  
  --use_peft True \  
  --lora_r 8 \  
  --load_in_4bit True \  
  --n_path_per_sample 10 \  
  --response_template "[/INST]"
```

Trains using SFT with LoRA adapters. Each training sample is a (question, reasoning-paths, answer) triple. Only the assistant response (paths + answer) is used for loss computation.

Chapter 11

CLI Reference

11.1 Global Arguments

Table 11.1: Arguments common across workflow scripts

Flag	Type	Default	Description
<code>-data_path</code>	str	rmanluo	HuggingFace dataset hub path/organisation
<code>-d / -d</code>	str	RoG-webqsp	Dataset name
<code>-split</code>	str	test[:100]	Dataset split (supports slicing)
<code>-predict_path</code>	str	results/GenPath	Output directory
<code>-force</code>	flag	False	Overwrite existing results
<code>-n</code>	int	1	Number of parallel processes
<code>-debug</code>	flag	False	Print debug output

11.2 Model Arguments

Table 11.2: Model configuration arguments

Flag	Type	Default	Description
<code>-model_name</code>	str	gcr-Llama-2-7b-chat-hf	Model name (determines registry lookup)
<code>-model_path</code>	str	meta-llama/...	HuggingFace model path
<code>-maximun_token</code>	int	4096	Max input token budget
<code>-max_new_tokens</code>	int	1024	Max tokens to generate
<code>-dtype</code>	str	bf16	Precision (fp32, fp16, bf16)
<code>-quant</code>	str	none	Quantisation (4bit, 8bit)
<code>-attn_implementation</code>	str	flash_attention2	Attention backend
<code>-generation_mode</code>	str	greedy	Decoding strategy
<code>-k</code>	int	1	Beam size / number of sequences
<code>-chat_model</code>	bool	true	Apply chat template

11.3 Decoding Strategies

Table 11.3: Supported decoding strategies

Mode	Behaviour	Typical use
greedy	Argmax at each step	Fast, deterministic baseline
beam	Beam search with <code>num_beams=k</code>	High-quality single path
sampling	Multinomial with temperature=0.6, top-p=0.95	Diverse paths
group-beam	Diverse beam search, <code>num_beam_groups=k</code>	Diverse high-quality paths
beam-early-stopping	Beam search with early stopping	Faster beam search

11.4 DCA-Trie Specific Arguments

Table 11.4: DCA-Trie workflow arguments

Flag	Type	Default	Description
<code>-dca_mode</code>	str	v1	v1 (static) or v2 (step-wise)
<code>-prompt_mode</code>	str	zero-shot	Prompt template (zero-shot , few-shot)
<code>-index_path_length</code>	int	2	Max DFS depth (k)
<code>-undirected</code>	bool	False	Treat graph as undirected
<code>-n_data</code>	int	None	Samples for FNR analysis
<code>-add_rule</code>	flag	False	Use pre-predicted rules
<code>-rule_path</code>	str	(long default)	Path to rule predictions

11.5 Training Arguments

Table 11.5: Training-specific arguments

Flag	Type	Default	Description
<code>-use_peft</code>	bool	False	Enable LoRA
<code>-lora_r</code>	int	8	LoRA rank
<code>-lora_alpha</code>	float	16	LoRA scaling
<code>-lora_dropout</code>	float	0.05	LoRA dropout
<code>-load_in_4bit</code>	bool	False	4-bit quantisation
<code>-n_path_per_sample</code>	int	10	Training paths per sample
<code>-response_template</code>	str	[/INST]	SFT response marker

Chapter 12

Frequently Asked Questions

12.1 Architecture and Design

1. **Why are there three trie implementations in `trie.py`?**

The pure-Python `Trie` class is the reference implementation. `MarisaTrie` wraps the `marisa-trie` C extension for production use (10-100× faster). The `DummyTrie*` stubs are unused placeholders from an earlier design iteration.

2. **Why does `get_registered_model` use substring matching?**

This allows the model name CLI argument to simultaneously serve as both the registry key and the human-readable label for output directories. `gcr-Llama-2-7b-chat-hf` contains "gcr" so it selects `GraphConstrainedDecodingModel`, and the full string is used for organising results.

3. **How does the trie interact with the LLM tokenizer?**

The trie stores token IDs, not tokens. Path strings are tokenised with `add_special_tokens=False` so there are no BOS/EOS tokens inside the paths. The `eos_token_id` marks the end of each complete path sequence. During generation, `GraphConstrainedDecoding.allowed_tokens_fn` strips the input prompt length from the prefix before looking up the trie.

12.2 Evaluation

1. **Why define both Accuracy and Hit?**

Accuracy measures how many of the expected answers were found (useful for multi-answer questions). Hit measures whether at least one correct answer was found (useful for questions with a single expected answer). The distinction matters for questions like "Name all children of Elvis Presley" where partial coverage should be rewarded.

2. **What does SIR measure that simple pruning ratio does not?**

Simple pruning ratio measures the number of paths removed. SIR formalises this into a normalised ratio and decomposes it into type-gate and range-gate components, enabling per-gate analysis. A high SIR_{type} with low SIR_{traj} indicates the type gate is doing most of the work.

3. **Why include FNR analysis?**

FNR validates that the symbolic gates are *safe* — they should never exclude a gold-truth path. A non-zero FNR would mean the oracle is over-pruning, potentially reducing recall even if precision improves. The experiment confirms 0% FNR for both gates on WebQSP.

12.3 Troubleshooting

1. **Model is not found in the registered_language_models, return HfCausalModel by default**

This message appears when the model name doesn't contain `gpt`, `gcr`, `proxy`, or others. The script falls back to `HfCausalModel`, which does not support the `generate_sentence` method with trie constraints. Either rename the model to include `gcr` or use a matching prefix.

2. **AttributeError: 'LLMProxy' object has no attribute 'prepare_model_prompt'**

`LLMProxy` originally lacked this method. It needs a pass-through that returns the input unchanged. This was fixed in the codebase.

3. **ValueError: The template name is not valid.**

The `prompt_mode` argument must be one of `zero-shot`, `mcq-zero-shot`, or `few-shot`. These map to prompt template attributes via string upper-and-replace (`zero-shot` → `ZERO_SHOT_PROMPT`).

12.4 Performance

1. **Why is Approach 3 faster than GCR?**

GCR builds a trie over all $O(\bar{d}^k)$ paths. DCA-Trie v1 filters paths before building the trie, reducing the trie size by 30-45%. DCA-Trie v2 never enumerates all paths — it expands incrementally, keeping only $O(\bar{d})$ paths in memory at any point.

2. **When should I use v2 instead of v1?**

Use v1 when memory for the full path set is acceptable and you want deterministic the same trie structure as GCR (for fair comparison). Use v2 when path count is very large ($k \geq 3$ or $\bar{d} \gg 20$) and memory is a concern.

3. **Can I run on CPU only?**

Yes. The symbolic `TypeOracle` (Approach 3) runs entirely on CPU. For model inference, use a small model (Llama-2-7B or smaller) with CPU offloading or quantisation.

Appendix: Quick Reference

12.5 File Function Map

Table 12.1: File function map

File	What to use it for
workflow/predict_paths_and_answers.py	Run GCR baseline constrained decoding
workflow/predict_symbolic_dca_trie.py	Run symbolic DCA-Trie (v1 or v2)
workflow/predict_final_answer.py	Run zero-shot or path-augmented answer prediction
workflow/finetune_kg_specialized_llm.py	Fine-tune a model with LoRA for KG path generation
workflow/build_graph_index.py	Pre-compute DFS path indices
src/qa_prompt_builder.py	Understand/modify prompt templates
src/trie.py	Understand/modify trie implementations
src/graph_constrained_decoding.py	Understand/modify token-level constraints
src/llms/base_hf_causal_model.py	Modify model loading or generation logic
approach3_symbolic/type_oracle.py	Modify/extend symbolic gates
src/utils/qa_utils.py	Understand/modify evaluation metrics
src/utils/graph_utils.py	Modify graph construction or path enumeration

12.6 Python Invocations Quick Reference

```
# GCR baseline
python workflow/predict_paths_and_answers.py \
  --model_name gcr-qwen2.5-3b \
  --d RoG-webqsp --split "test[:10]" \
  --index_path_length 2 \
  --generation_mode group-beam --k 5

# DCA-Trie v1
python workflow/predict_symbolic_dca_trie.py \
  --model_name gcr-qwen2.5-3b \
  --d RoG-webqsp --split "test[:10]" \
  --dca_mode v1 --index_path_length 2 \
  --generation_mode group-beam --k 5

# DCA-Trie v2
python workflow/predict_symbolic_dca_trie.py \
  --model_name gcr-qwen2.5-3b \
  --d RoG-webqsp --split "test[:10]" \
  --dca_mode v2 --index_path_length 2

# Zero-shot baseline (via proxy)
python workflow/predict_final_answer.py \
  --model_name proxy-qwen2.5-3b \
  --d RoG-webqsp --split "test[:10]"

# FNR analysis only
```

```
python workflow/predict_symbolic_dca_trie.py \  
  --model_name gcr-qwen2.5-3b \  
  --d RoG-webqsp --split "test[:10]" \  
  --dca_mode v1 --n_data 100
```